

Review: Linear Structure

Fei Zuo

Department of Computer Science
University of Central Oklahoma

Outline

- Array and string 
- Queue and stack 
- STL containers 

Linear Structure

- In brief,
 - A sequence of items
 - Order matters

“A collection of items ordered by a single property so that each item, except possibly for the first or last, has a unique predecessor and a unique successor”.

Examples: array, string, stack, and queue

Array

- A list of elements
- Each element must be in the same type
- The size of an array cannot be changed anymore once it is defined
- Questions:
 - How to traverse an array?
 - How to find the max element in an array?
 - How to update the value of one element?
 - How to copy an array?

String

- A sequence of characters.

```
#include <iostream>
#include <string>
#include <cstdlib>

using namespace std;

int main(int argc, char* argv[]) {
    string s1 = "first"; // Initialization
    string s2;
    s1 += " string"; // Concatenation
    s2 = s1; // Assignment
    cout << s1 << endl;
    cout << s1.length() << endl;
}
```

String: advanced operations

- *erase*: Removes a sequence of characters from a string
- *find*: Searches a string for the occurrence of another string
- *substr*: Returns, as a string, part of another string
- *replace*: Replaces a substring of characters with another string
- *insert*: Inserts a string into another string

String: advanced operations: examples

```
string s1("Demonstrating all the advanced");  
string s2("string functions!!!");
```

```
// erase the exclamation marks  
s2.erase(16, 3);
```

```
// replace 'all the' with 'some'  
s1.replace(14, 7, "some");
```

```
// insert a space at the beginning of s2  
s2.insert(0, " ");
```

```
// extract everything after "some" from s1 into  
index = s1.find("some");  
string s3 = s1.substr(index);
```

STL Container: vector

- In C++, STL (*Standard Template Library*) provides general-purpose components for common programming tasks.
- The library is organized into three main abstractions.
 - Containers
 - Iterators
 - Algorithms

STL Container: vector

■ Define

```
vector<int> v1;  
vector<vector<int>> matrix;
```

■ Initialize

```
vector<int> v2(5);           // 5 elements, initialized to 0  
vector<int> v3(10, 1);      // 10 elements, initialized to 1  
vector<int> v4(v3);         // v4 is a copy of v3  
vector<int> v1;             // initially empty
```

■ Two operations

```
push_back()  
pop_back()
```

STL Container: vector

■ Access element

```
v[1] = 2;  
v.at(2) = 45;  
v.front() = v.back();
```

■ Using iterator

```
for (forwardIterator = result.begin();  
forwardIterator != result.end(); forwardIterator  
cout << *forwardIterator << " ";  
)  
  
for (backwardIterator = result.rbegin();  
backwardIterator != result.rend(); backwardIterator++){  
cout << *backwardIterator << " ";  
}
```

STL Container: deque

- An object of type deque can store and provide access to a linear sequence of elements. In this respect, a deque is similar to a vector.
- In fact, the two classes share nearly identical class interfaces.
- But, class deque and vector differ in their implementation.

STL Container: deque

```
#include <deque>
#include <iterator>

int main(int argc, char* argv[]) {

    deque<int> d(10, 1);
    d[9] = 2;

    cout << d.front() << " " << d.back() << endl;
    d.push_front(5);
    d.pop_front();
    d.push_back(3);
    d.pop_back();

    cout << d.size() << endl;

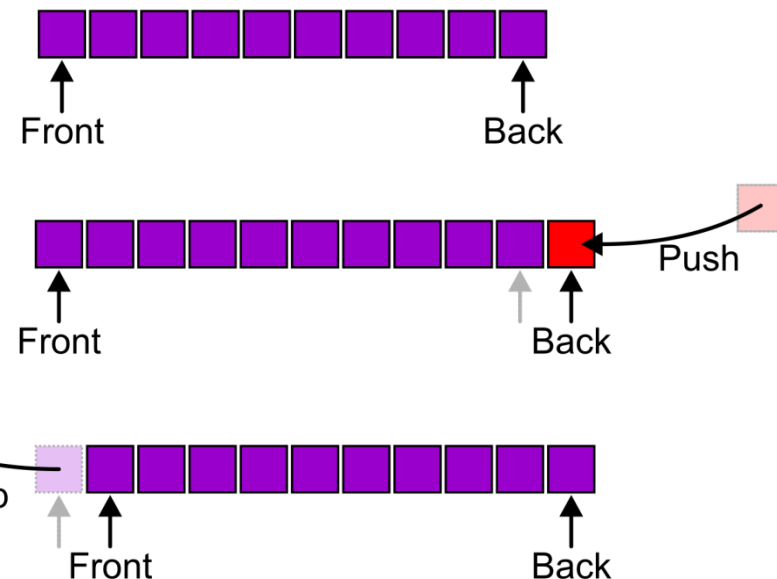
    ostream_iterator<int> out(cout, " ");
    copy(d.begin(), d.end(), out);

}
```

Queue: first-in, first-out (FIFO)

- A first-in, first-out element insertion and removal policy: the first element added is the first element removed.

- Put another way, things are only removed from the beginning and only added to the end of queues.



- A real world example of a queue data structure:
 - ❑ a line at a movie theater
 - ❑ a "First Come, First Served" sign

STL: queue adapter

- The STL contains *adapters* that provide a new, but similar interface to an existing container.
- The queue adapter provides the interface suitable for a queue data structure. It is based on the deque container.

queue member	deque equivalent
push	push_back
pop	pop_front
front	front
back	back
size	size
empty	empty

queue member functions and their deque equivalents

STL: queue example

```
#include <cstdlib>
#include <queue>

int main(int argc, char* argv[]) {

    queue<int> q;

    // push and pop
    q.push(1);
    q.pop();

    // front and back
    q.push(1);
    q.push(2);
    cout << q.front() << endl;
    cout << q.back() << endl;

    // size and empty
    cout << q.size() << endl;
    cout << q.empty() << endl;
}
```

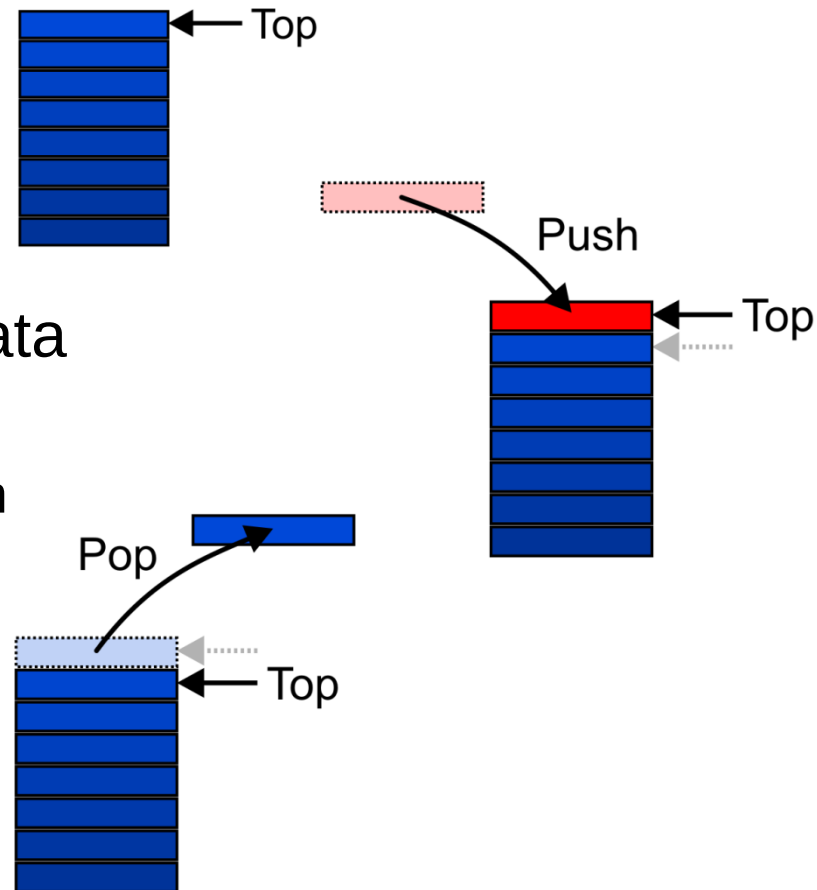
Stack: last-in, first-out (LIFO)

- "Last-in, First-Out" element insertion and removal policy:

- Items can only be added at and removed from one end of the sequence.

- A real world example of a stack data structure:

- a stack of plates or dishes in a kitchen



STL: stack adapter

- The STL stack adapter provides an interface suitable for use as a stack

```
#include <cstdlib>
#include <stack>
```

```
int main(int argc, char* argv[]) {
```

```
    stack<int> s;
    // push and pop
    s.push(1);
    s.pop();
```

```
    // top
    s.push(10);
    s.push(11);
    cout << s.top() << endl;
```

```
    // size and empty
    cout << s.size() << endl;
    cout << s.empty() << endl;
```

```
}
```

Stack: problem solving example

■ Valid Parentheses

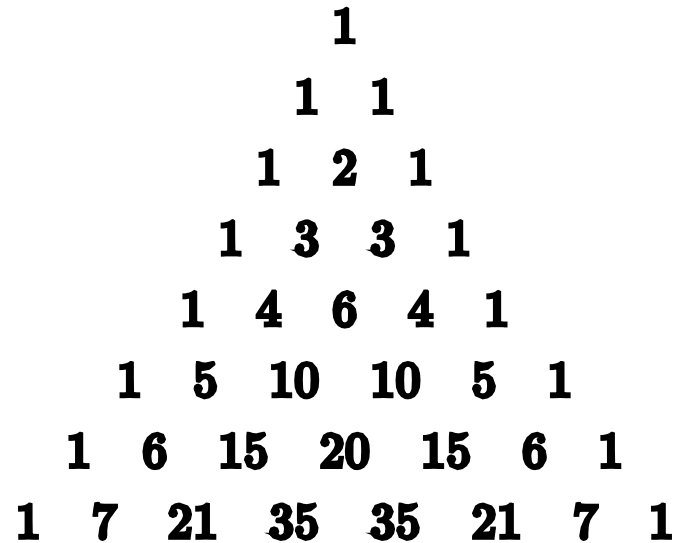
- ❑)(
- ❑ ((()))
- ❑ ()
- ❑ (a)((b)((c)(d)))

Given a string *s*, please determine whether the parentheses are used correctly?

Another similar example: Leetcode 20

Queue: problem solving example

■ Pascal's Triangle



A diagram showing the first eight rows of Pascal's triangle

*https://en.wikipedia.org/wiki/Pascal%27s_triangle

Another similar example: Leetcode 118, 119